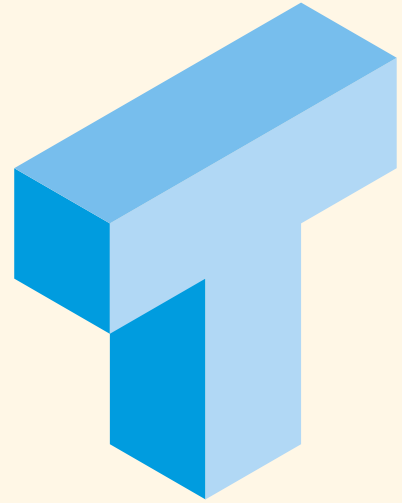
Everything you need to know about Temporal tables

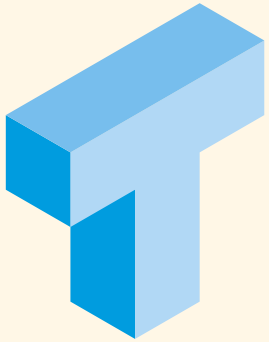
…or at least enough to get started

Magnus Ahlkvist
He/Him



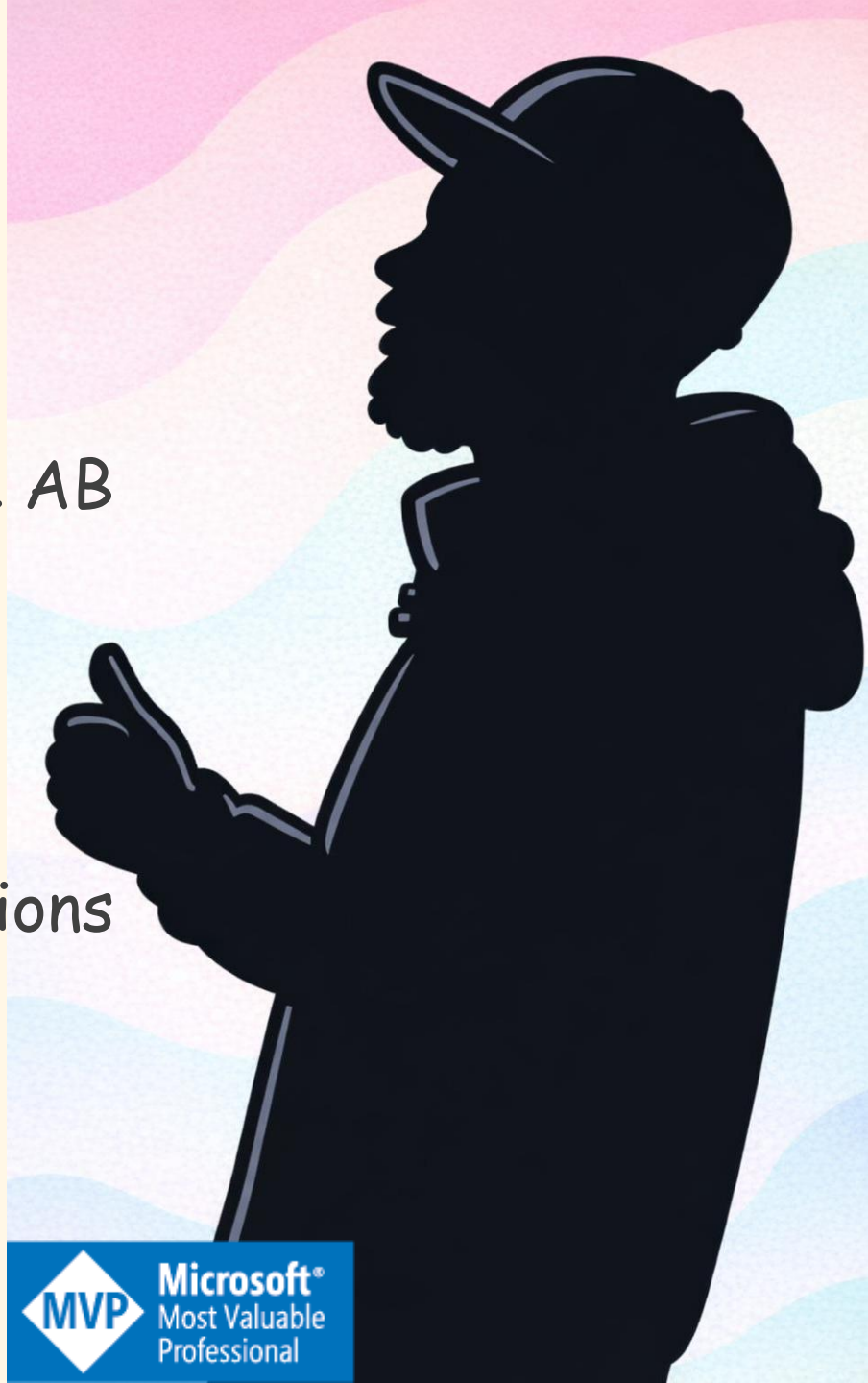
Session contents

- What are temporal tables and why should I care
- Create a new temporal/system-versioned table
- Make an existing table system-versioned
- Migrate another temporal pattern to system-versioned
- Some performance considerations
- What I'm want in addition to all this



About Magnus

- SQL Server consultant Transmokopter SQL AB
- Data Platform MVP
- Social media: @transmokopter
- E-mail: magnus@transmokopter.se
- github.com/transmokopter - repo Presentations
- He/Him

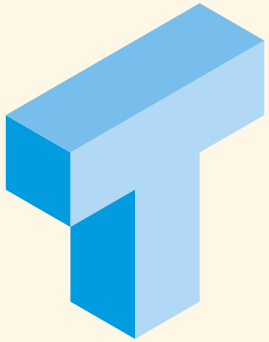


What are temporal tables and why should I care?

- Temporal tables = system-versioned tables
- Automatic versioning of rows in a table
  - Old value is moved into history table when a row is updated
  - Old value is moved into history table when a row is deleted

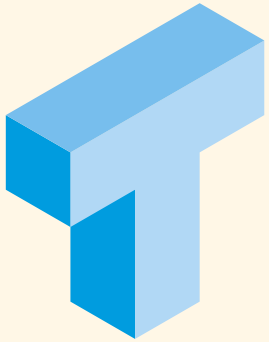
…and why should I care?

- We need to maintain an audit-trail
- Or we need to know what the price was for a product at a specific point in time
- System-time properties of a table can be used for loading SCD2 dimensions in a Data Warehouse



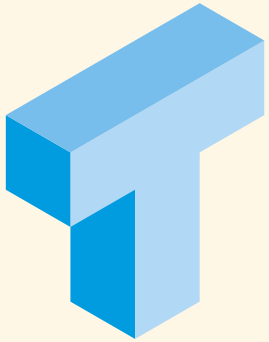
Three scenarios

- A new table with versioning
  - "Easy" – just create the table as system versioned
- An existing table that you want to start versioning
  - "Less easy but not really too hard". Create the From- and To-columns, give them default values and then start versioning.
- An existing table versioned with some sort of home-cooked temporal pattern
  - "More complex, implementation requires downtime". Alter existing columns to datetime2(7). Create history table. Move history rows to history table. Set new To-value, if it's not already 9999-12-31 23:59:59.9999999.



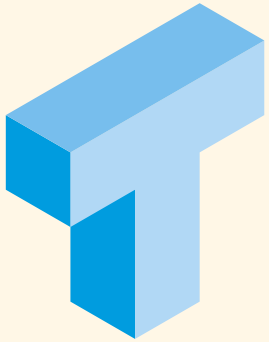
Gotchas

- Selecting historical records means accessing both the "current" and the history table.
  - Look at execution plans, figure out indexing patterns for your history table
- Everything is duplicated. The whole row, not just the changes.
  - 80 columns in a table, two or three columns that can ever change => Probably worth reworking your data model
- Purging historical records
  - Set HISTORY_RETENTION_PERIOD when you enable system versioning
  - Or (probably more effective for you) – partition the history table and switch out the oldest partition in a scheduled job



How does it work?

Demo



How about business-time?

- Business time dimension
  - Price is updated. But ValidFrom is not when the row was physically changed
- DB/2 has had this since long
- Would be more useful for SCD2
  - … and lots of other scenarios